

网络篇-1

一、网络规划设计及模拟

2025.01-2025.02

中小型企业网络规划设计及模拟

负责人

- **基本架构**：基于“接入+汇聚+核心+出口”架构，集成安全策略，实现各部门的网络互通与数据安全防护；
- **初步规划**：规划 VLAN，分隔各部门网络，再配置ACL使得管理部门特权访问，部署DHCP服务，实现IP自动分配，提高管理效率，通过OSPF协议实现路由可达；
- **安全稳定**：防火墙策略控制内外网访问，使用NAT策略隐藏内网IP地址，无线网络采用WPA2加密及SSID与业务VLAN绑定，保障通信安全；核心层采用 MSTP多实例设计防环，部署 VRRP 双机热备+链路聚合，实现网关冗余和负载分担；
- **网络优化**：通过OSPF动态路由+BFD毫秒级检测联动，网络收敛时间从最低59ms缩短至最低14ms
- **项目成果**：结合防火墙、负载均衡设备的日志分析，业务系统可用性至99.9%，实现内网威胁事件防御成功率98%

1、项目思路

第一：确定架构

考虑到企业未来 3-5 年的扩展性以及故障域隔离，我没有采用扁平化结构，而是敲定了经典的‘接入-汇聚-核心-出口’四层架构。这奠定了整个网络的物理基座，让接入层管准入，汇聚层管策略，核心层管高速转发。

第二：VLAN划分

骨架搭好后，开始做逻辑隔离。通过规划 VLAN 把研发、财务等不同业务部门划分到不同的局域网，并在汇聚层集中部署 DHCP Relay 统一下发 IP。为了让这些房间能互相通信，我在底层跑了 OSPF 动态路由协议，打通了全网的基础可达性

第三：高可用设计

核心层引入了 MSTP 多实例与 VRRP 的黄金组合。这不仅消除了二层环路，更实现了核心交换机的双机双活与网关冗余，保障了企业网的生命线。

第四：安全与权限控制

引入了企业级防火墙，把网络划分为 Trust（内网）、DMZ（服务器）和 Untrust（外网）三大安全区域。利用状态检测机制和自上而下的安全策略，精确管控进出流量；同时部署 Easy-IP（NAT），隐藏了内网的真实拓扑，实现安全收口

第五：优化

因为传统的 OSPF 故障感知太慢，我叠加了 BFD 毫秒级检测联动，将原本几秒甚至几十秒的收敛时间，压缩网络收敛时间，最终，配合负载均衡和 Syslog 日志的联合监控

2、网络架构

接入层：提供高密度的端口接入，将终端连进网络，只做纯二层转发

汇聚层：承上启下，收敛接入层的流量，执行策略，汇聚交换机在这里把二层的 MAC 转发，转换成了三层的 IP 路由，简历里提到的“配置 ACL 使得管理部门特权访问”，最标准的部署位置就是汇聚层。把 ACL 挂在汇聚层，能让非法的跨部门访问流量在基层就被丢弃，绝不让脏流量去消耗核心交换机的 CPU

核心层：极致的块，只负责高速转发。这里结合了MSTP + VRRP + 链路聚合 的架构，是为了在模拟环境中验证和巩固我对于传统二三层高可用协议底层运作逻辑的掌握

出口：内外网的绝对隔离，以及地址转换。部署企业级防火墙。做 NAT 把内网私有 IP 转换成公网 IP；划分安全区域

3、关键技术

吃透NAT

静态 NAT，将 1 个私有 IP 固定绑定到 1 个公网 IP 上，它需要独占一个公网 IP 供外部访问
动态 NAT，多对多，如果地址池里面只有10个IP，第 11 个人想上网，发现池子空了，他就上不了网了，极其不可靠

NAPT端口地址转换：内网成百上千台机器，共享同一个公网 IP，路由器不仅转换 IP，还转换 TCP/UDP 的源端口

云原生视角：

SNAT：内网主动访问外网，业务代码跑在 Docker 容器（IP 172.17.0.2）里，它要调用外部微信支付的 API。请求经过宿主机网卡时，宿主机做了SNAT，把源IP变成了宿主机的局域网或公网IP，但是容易端口耗尽

DNAT：外网主动访问内网，数据包进入网关时，将其目标 IP（通常是公网IP或LB的IP）修改为内网真实的业务服务器IP

吃透ARP

二层通信中，数据帧的源目的MAC地址是不会变的

ARP缓存表：存储设备本地的IP-MAC映射记录，减少重复解析开销，TTL结束后自动删除

主机A发送数据给主机B，整个工作流程：

1. A检查本地ARP表，若没有IP-B的映射，就像局域网内所有设备发送ARP广播包
2. 广播包的目的地MAC地址全F，B收到后发现IP是自己，返回单播ARP应答包，携带有自己的MAC地址
3. A收到来自B的应答之后，把B的MAC地址映射到ARP缓存表中

免费ARP：主动发送ARP更新请求，用于检测IP地址冲突

代理ARP：主机不在同一子网，网关代替目标主机回复ARP请求

ARP欺骗：篡改MAC地址，截取数据帧

吃透vlan

VLAN作用：分割广播域，避免广播风暴，造成网络资源的浪费，可以灵活的组网，便于管理，同时还有安全加固的功能

三层端口是不接受/发送带标签等数据，但是像路由器的子接口是可以的（单臂路由）

Access：端口VLAN 和 标签 VLAN ID 是相同的

接收数据：接收到的数据不带VLAN标签，端口就会自动给该数据打上 VLAN 标签；接收到的数据有标签，检查标签ID是

跟端口VLAN ID相同，相同则接收，不相同就丢弃

发送数据：发送时会剥离标签

Trunk：首先要设置端口VLAN ID 和 允许放行的ID

接收数据：接收到的数据带VLAN标签，检查是否在可放行ID列表中；接收到的数据不带标签，打上本端口的VLAN ID，然后再看是否允许放行；（如果配置出错，会导致本端口ID不在可行ID列表中）

发送数据：发送的数据的VLAN标签与端口VLAN 相同，剥离标签发送（Trunk）；与端口VLAN不同，就保留标签发送

如果数据帧需要发送到不同VLAN（例如VLAN20或VLAN30），三层交换机会执行VLAN间路由。这意味着交换机会根据其路由表来决定如何将数据帧转发到目标VLAN。这通常涉及到以下步骤：

1. 去除VLAN标签：交换机首先去除原始数据帧的VLAN标签
2. 查找路由表：交换机查找其路由表，确定如何到达目标VLAN（网段和VLAN ID和进行绑定）
3. 添加新的VLAN标签：如果目标VLAN与源VLAN不同，交换机会为数据帧添加新的VLAN标签
4. 转发数据帧：交换机将数据帧转发到目标VLAN的相应端口

吃透DHCP

先来看看DHCP的工作原理

- 1.发现：设备发送广播包寻找DHCP服务器（也可以是路由器，交换机）
- 2.回应：很多服务器都会收到该数据包，单播回应客户端
- 3.请求：实际上，客户端会收到很多的回应报文，然后选出最适合自己的IP地址，广播回应所有服务器，表示自己已经找到了最适合自己的IP，其他服务器就不需要再提供给我另外的IP地址了
- 4.确认：服务器确认分配，回应确认报文

核心考点：来说说DHCP中继

面试官问：“你的网络里分了多个 VLAN（比如财务部 VLAN 10，研发部 VLAN 20），而 DHCP 的 Discover 报文是广播包，广播包是无法跨越 VLAN 和路由器的。如果你的 DHCP 服务器集中部署在核心交换机上，财务部的电脑是怎么拿到 IP 的？”

解决方案：在汇聚层几交换机配置DHCP中继（项目里面，是核心层交换机作为网关配置DHCP，所以不需要DHCP中继，在通常的网络设计中，应该有专门的DHCP服务器，然后在网关处配置DHCP中继，网关极其聪明，它会把二层广播包拦截下来，重新封装成一个三层单播包，直接精准地发送给核心 DHCP 服务器的真实 IP）

当面试官问到：“你的 DHCP 是怎么部署的？有没有用到中继？”（我觉得最好回答用了，体现高度）

“在最初规划这个中小型企业网时，为了简化架构、节约服务器成本，我是直接在核心交换机上开启了 DHCP 服务。因为各部门的网关（VRRP 虚拟 IP）本来就在核心层，所以员工的 DHCP 广播包直接抵达核心层处理，不需要配置 DHCP 中继。

但考虑到未来的扩展性，如果在二期项目中，公司引入了专门的机房和 Windows/Linux 服务器群落，我会将 DHCP 服务迁移到独立的服务器上集中管理。到那时候，我只需要在核心交换机的各个部门网关上开启 DHCP Relay（中继代理）功能，将广播包单播指向新服务器即可，底层的接入和汇聚层完全不需要做任何改动。”

吃透防火墙策略

防火墙是默认拒绝，只有明确允许的，才能通过

Trust（信任区）： 极高安全级别（通常值为 85）。包含公司内部的所有终端网络

Untrust（非信任区）： 极低安全级别（通常值为 5）。直接连接外部互联网

DMZ（服务器）： 中等安全级别（通常值为 50）。如果你们公司有对外提供服务的官网服务器、邮件服务器，绝对不能放在 **Trust** 区，必须放在 **DMZ** 区。这样哪怕服务器被黑客攻破了，黑客也无法以此为跳板直接攻击内网

说一下ACL和防火墙的区别？

路由器的 ACL 是无状态的： 员工访问百度，你在出方向放行了 HTTP 请求。但当百度的网页数据包返回时，路由器不认识这是回包，如果入方向没有明确放行，包就会被丢弃。这就要求网工必须配双向 ACL，极其繁琐且极不安全

防火墙是基于状态会话的： 当Trust发起访问百度（Untrust）的请求时，只要防火墙策略允许出站，防火墙就会在内存里动态建立一张‘会话表’。当百度的回包抵达防火墙时，防火墙一查会话表：“哦，这是里面员工主动请求的数据”，直接放行回包，不需要在 Untrust 到 Trust 方向额外写放行策略”

本项目中，在防火墙部分使用了Easy-IP？（端口复用技术）

当内网的张三（192.168.61.2）和李四（192.168.62.2）同时访问百度时：

出站： 数据包到达防火墙，防火墙一看匹配了 Easy-IP 规则。它会毫不留情地把张三和李四数据包里的源 IP全部抹掉，替换成 GE1/0/4 接口的公网 IP 200.200.200.1，防火墙在转换IP的同时，会给他们分配不同的“源端口号”并建立映射表，当百度的回包抵达防火墙时，防火墙一查，就可以把目标 IP 和端口换回私网地址，精准扔回给内网

吃透BDF

面试官发难：OSPF自己不是有 Hello 机制吗？断网了OSPF自己难道发现不了吗？

在以太网中，OSPF默认 10 秒发一次 Hello，40 秒收不到才宣告邻居死亡。如果是中间的传输设备坏了，接口物理状态没 Down，OSPF 就要傻等 40 秒才会切换路由。这 40 秒在现代业务中意味着灾难性的断网。如果把 OSPF 的 Hello 时间改成 1 秒，Dead 时间改成 3 秒。但在大型网络中，让路由器的 CPU 每秒处理成百上千个 Hello 包，会直接把 CPU 跑满挂掉。

OSPF和BDF的联动机制

1. OSPF 进程在启动并建立邻居后，会主动向 BFD 模块‘注册’，告诉 BFD：‘帮我盯着那个 192.168.1.2 的邻居。’
2. BFD 模块立刻在底层以极快的速度（如 10ms 间隔）和对端建立 BFD 会话
3. 一旦链路中间发生故障，BFD 在连续 3 个包（30ms）没收到后，立刻向上层 OSPF 控制面发送一个信号
4. OSPF 收到信号后，立刻跳过那漫长的 40 秒 Dead Timer 等待，瞬间撕毁邻居关系，重新运行 SPF 算法，将流量切换到备用链路上

缺点：

网络抖动导致误杀，全局路由重新计算，假死
不管是 CPU 负责，还是底层芯片负责，都有开销

面试官非常可能问的：“你简历上写，收敛时间缩短至最低 **140ms**。请问这 14ms 是怎么测出来的？现网中，光是 OSPF 重新计算一次路由（SPF 算法）再加上把路由下发到硬件转发表（FIB），可能都不止 14ms，你的 BFD 参数是怎么配的？”

这个 140ms 是我在 eNSP 模拟器环境下，通过高频持续 Ping（发包间隔极小）配合 Wireshark 抓包，将结果发给AI提取的，主要是体现收敛速度提升

在真实情况下，如何去测量收敛时间呢？

- 1、硬件打流仪
- 2、路由器自带的探针技术
- 3、在接收端开启 Wireshark 抓包，或者直接看 Ping 命令结束后的丢包统计。结合丢包数量和发包间隔，粗略估算断网时间。虽然受限于操作系统 CPU 调度，精度远不如硬件打流仪，但证明网络切换时间在 50ms 以内是足够的。

单向BDF的问题（重要）

条件不足：设备版本不支持，跨部门，跨企业权限隔离，异构网络对接等等

在条件不足的情况下，设备 A 想探测链路，但设备 B 不配合。于是，设备 A 构造出一个特殊的 BFD 探测包：它的源 IP 是 A 自己，目的 IP 也是 A 自己，这样，B收到探测包之后，就会转发给A，设备 A 只要能收到自己发出去的这个“回声（Echo）包”，就知道从 A 到 B 的这根网线是通的！

我觉得这块很重要，之前滴滴面试问过，A设备这种欺诈行为，更加体现了网络的复杂性和灵活性

HTTP详解

HTTP/1.1 有什么性能瓶颈？HTTP/2 是怎么解决的？为什么后来又搞出了 HTTP/3？

HTTP/1.1 规定同一个 TCP 连接上的请求必须按顺序处理。即使启用了管道化（**pipelining**），响应也必须按请求顺序返回。如果前一个请求的响应被阻塞（例如处理时间较长），后续所有请求的响应都会被阻塞，导致整个连接上的资源无法并行加载

HTTP/2 使用乱序碎片化处理，解决了队头阻塞的问题

HTTP/2 虽然解决了应用层的阻塞，但底层依然是 TCP。只要 TCP 丢了一个包，整个连接都会停下来等重传，所以 HTTP/3 直接废弃了 TCP，底层改用基于 UDP 的 QUIC 协议，实现了真正的和无阻塞传输

HTTPS的加密

混合加密：

1. **TLS握手阶段（非对称加密）**：用非对称加密解决对称密钥的安全分发，同时完成服务器身份认证，不怕窃听、冒充，【高安全】（**RSA、ECC**）
2. **数据传输阶段（对称加密）**：加密海量**HTTP**数据，保证传输效率，支撑高并发【高性能】（**AES**）
过程（注意中间人的攻击）：服务器本地生成「私钥 + 公钥」对 → 公钥通过「数字证书」安全分发 → 客户端用服务器公钥加密「预主密钥」→ 服务器用私钥解密 → 双方基于预主密钥推导「对称会话密钥」→ 后续数据用对称密钥加密

HTTP状态码

状态码：

1XX，连接已经建立，正在交换协议

2XX，成功，业务正常，**200**是常态

3XX，重定向，**301/302**涉及域名跳转

4XX，客户端出现错误。**403**权限拒绝，**404**资源不存在

5XX，**500**代码逻辑错误，**502**网关异常（后台进程挂了），**504**后端响应超时（服务器端出现错误）

502 bad gateway，后端死了或拒绝沟通

504 Gateway Timeout，**Nginx** 成功把请求发给了后端，但是等啊等，等到了 **Nginx** 配置文件里设定的最大超时时间

499 后端太慢了，客户端等不及了，主动把 **TCP** 连接给断开了，有时候不属于客户端的问题

数据化成果

业务可用性达到了 99.9%。网络工程里常说的‘几个 9’是怎么算的？

一年有 365 天 * 24 小时 = 8760 小时

意味着一年内因为网络故障导致的核心业务中断时间，被严格控制在了 9 个小时以内。

你是用什么看日志的？

在项目中，我部署了一台集中的 **Syslog** 服务器，我让防火墙和负载均衡把日志全部实时推送到中心节点。当发生故障时，拉出那个时段的日志信息，就能快速定位

防御成功率99%

坦白地说，这个项目是在 **eNSP** 等虚拟化环境中构建的，并没有真实的公网黑客流量

简历上写这个 **98%**，其实是我在做这个企业网安全规划时，给这套‘**ACL + 防火墙区域隔离 + NAT**’组合拳设定的安全架构设计预期指标

我在 **Untrust** 区域模拟了外部源 **IP**，尝试去 **Telnet** 甚至 **SSH** 我们的核心路由器和内网网段，全部被防火墙的 **Trust/Untrust** 安全策略静默丢弃

至于为什么不写 **100%**，是因为我非常清楚，仅靠底层的交换机和传统的四层防火墙策略，是防不住应用层的零日漏洞

要做到**100%**，离不开更高端的设备和先进的技术，这也会带来一定的成本

4、三大平面

管理平面：负责设备与“人”的交互。它不关心具体的数据怎么走，只关心设备的健康状态和配置指令

控制平面：它在后台默默运行复杂的算法，计算出全网的最优路径，并生成路由表（**RIB**）、**MAC** 地址表、**ARP** 表

数据平面：专注、极其迅速地**执行搬砖任务**。它没有脑子，绝不自己算路由，只看控制平面发给它的“最终派件单”（硬件转发表 FIB）

5、常见攻击

DNS反射放大攻击，伪造IP

DNS欺骗，中间人攻击，将用户骗到钓鱼网站

DNS劫持，黑客盗取你在域名注册商的管理账号，将权威服务器改成黑客服务器

二、网络自动化项目实施

2025.07-2025.08

园区网络自动化项目实施

负责人

- **项目要求**：企业内网需实现内部网段互通、安全远程管理，并支撑业务 VLAN 隔离、压力测试，保障业务稳定运行，通过**自动化**运维降低人工成本、缩短故障定位时间，为核心业务持续运行提供网络支撑；
- **技术实施**：开启设备SSH功能，编写**Python**脚本，调用**Netmiko**库连接设备，创建业务VLAN，并且利用**TextFSM**工具，处理ARP表回显内容，转换为CSV格式存储，用于分析**IP-MAC**映射及网络拓扑变化，搭建**Linux虚拟机**日志服务器，通过Syslog采集系统日志，使用**正则表达式**提取关键信息，在虚拟机中部署**Cron**定时执行py文件，便于及时发现异常信息；
- **项目成果**：配置自动化下发覆盖**90%** 以上的重复性操作，ARP 采集、日志分析等核心运维环节实现**100%自动化**，基于ARP 结构化数据与日志**实时监控**，故障定位耗时下降 **50%**

1、Netmiko函数库

为什么选择netmiko而不选择Paramiko？

1. 有些命令的输出显示太长，也就是回显不完整，**Paramiko** 傻傻等不到完整回显，脚本直接卡死。而**Netmiko** 极其聪明，它会自动识别网络设备的翻页符，并在后台默默帮你敲空格，直到把所有文本一次性拉取下来
2. 连上设备后，要进特权模式（**Enable**）或系统视图（**System-view**）。用原生库，我得自己写正则去匹配 `>` 还是 `#` 提示符。**Netmiko** 自带了 `enable()` 和 `config_mode()` 方法，完美封装了各大厂商的提权逻辑

你的代码是怎么写的？（代码思路）

1. **ConnectHandler**用来和设备建立 **SSH** 通道，传入一个包含 **IP**、账号、密码、设备类型（**device_type**）的字典
2. `send_command()` 查询命令，`send_config_set()`下配置
3. 如果遇到设备死机或密码错误，原生脚本会直接抛出报错并终止运行，导致后面的设备全部漏配，**try...except** 语句。专门捕获 **Netmiko** 自带的两个致命异常，可以使得在发生错误时脚本不会崩溃，继续执行（多线程并发执行）

更大局观的说法：

1. 模块化设计，构建基础设备信息字典模板，如果后续修改信息，直接在模板里面修改就好了
2. 将配置命令单独写进一个文档，使用**netmiko**工具，逐行送入命令，这也是选择**netmiko**的原因
3. 因为网络环境不可靠，加入**try...except** 块中，保障代码运行的健全性
4. **ARP**回显是无规则的纯文本字符，使用**testFSM**提取回显信息，并转换成**csv**存储（作为一个简单日志记录）
5. 搭建了**syslog**服务器，**cron**定时任务触发**python**脚本

2、textfsm的作用

面试官：Python 原生就有 `re` 正则模块，也能做字符串截取。你引入 TextFSM 这个额外的第三方库，是不是把简单问题复杂化了？

网络设备的CLI输出是给人看到，而不是机器看的，一旦设备升级了系统版本，表头多了一个空格、或者多了一列无关紧要的数据，我写的原生正则就会全盘崩溃

TextFSM 完美解决了这些痛点。它本质上是一个‘文本状态机’，它把繁杂的提取逻辑从 **Python** 核心代码里抽离了出来，变成了一个独立的‘模板文件’

模板里会写需要匹配的关键信息，并且它会一行一行地读取的输出。只要扫描到符合匹配的行格式，就会给出我所定义的标准格式

（现实中，很多轮子都已经造好了）

真实工程中，开源社区的 **ntc-templates** 库。它内置了全球网工贡献的成百上千种主流设备模板。更棒的是，**Netmiko** 在底层已经集成了它！我只需要在发送命令时加上 `use_textfsm=True` 参数，**Netmiko** 就会在后台自动匹配模板并解析，直接把字典数据返回给我

3、日志部分

面试官：你是如何采集日志的？

我直接利用了 **Linux** 系统自带的 **rsyslog** 服务

我给所有交换机下发了指令，将**syslog**报文发往那个日志服务器，并打开日志服务器的端口监听报文

面试官：了解过日志架构吗？

ELK 则是目前全球互联网大厂（包括阿里、腾讯、字节）都在用的正规军

Logstash：专门负责收集、过滤、转换数据的管道，可以非常强悍地把交换机吐出来的非结构化 **Syslog**，清洗成极其标准的 **JSON** 格式

Elasticsearch：一个基于 **Lucene** 的分布式全文搜索引擎/数据库，倒排索引技术，实现更快定位

Kibana：可视化 **UI** 界面，瞬间把那个频繁爆破的源 **IP** 生成一张热力图、饼图、折线图，更加方便查找问题

Filebeat -> Kafka -> Logstash -> Elasticsearch -> Kibana

三、应用场景题

面试官问：我们说“网络请求很慢”的情况，可能是哪些原因导致的

客户端：客户端把域名解析成 **IP** 经历了太长的路由，或者本地**DNS**缓存失效，或者是路由抖动，丢包，**TCP** 一直重传

接入层（网关，负载均衡）：**LB**并发极高，连接数被耗尽，新的请求就会排队，或者被拒绝

计算层：**CPU**、内存、磁盘 **I/O** 达到极限，导致请求在服务器队列中等待处理；代码逻辑效率低等等

存储层：数据库缺乏正确的索引，导致全表扫描；或者高并发下触发了严重的行锁/表锁等待；正在疯狂读磁盘

面试官问：某台服务器出故障，排查思路

先止血，后排查，底自顶向下，最终闭环

1. 立刻把它从 **Load Balancer** 或 **K8s Service** 的后端池里摘除（隔离），或者利用 **Ansible** 迅速拉起一台新机器顶替，让用户的流量先流向健康的节点。把故障现场‘保护’起来后，再进行后续的深度排查
2. 看监控大屏，是否有时间节点发生异常，锁定故障时间范围
3. **SSH**登陆，测试连通性，进行资源层排查，负载>内存>磁盘>网络
4. 查看相关日志，先错后访，先实时后历史，先关键字后全文
5. 复盘，补充新监控规则并验证

面试官问：你做网络项目时，发现ping不通，该怎么排查？

拓扑连接、控制平面、数据平面三个层次展开
检查设备连线、检查接口状态、检查 **VLAN** 配置；
查看路由表、检查动态路由协议状态、检查 **ARP** 表；
检查 **ACL** 和防火墙策略、使用抓包工具

四、绿盟实习

为什么要给 IDS 导入证书？

绿盟的 **IDS** 等安全设备通常是通过 **web** 控制台进行管理的。为了防止管理员在输入账号密码时被抓包窃听，**web** 服务必须开启 **HTTPS**。
我导入的通常是由企业内部 **CA** 签发、或者向权威机构申请的 **SSL/TLS** 服务器证书。这样管理员在浏览器访问设备 **IP** 时，就不会报‘证书不受信任’的红色警告，满足了等保合规要求
二就是抓https里面的加密流量。**IDS**如果没有私钥，抓到的都是乱码，无法检测

IDS 是旁路镜像还是串联拦截？

探针--分析流量
绿盟的 **IDS** 绝对是旁路部署的，路部署的最大好处是绝对不影响业务主链路

遇到的问题

交接过来的资产表密码全错，甚至有很多机器重装后，通过远程连不上 **SSH**
由于不知道系统密码，我直接通过 **BMC** 带外管理进入控制台，进入单用户模式，强行重置了 **root** 密码
排查 **22** 端口连通性：查看是否开放该端口，先看网卡状态，**ss -tulnp**看看sshd服务是否开启，查看是否有防火墙拦截

干什么工作内容？

网络边界网关部署
参与机房深夜割接，通过修改物理拓扑或路由走向，将业务流量串联引流到绿盟下一代防火墙
直接访问**ssh**，在防火墙上规划网络逻辑，清晰定义 **Trust**（内网）、**Untrust**（外网）和 **DMZ** 区，并配置对应的静态路由
提前准备一个安全模板，直接省去**URL**过滤、防病毒扫描这些耗时内容，提高效率

国产化操作系统，并安装**vids**
收集重装前设备参数：**cpu**，内存等等
重装系统
按照信息，安装依赖环境，搭建网桥（虚拟了一台交换机），目的是让后面创建的虚拟机（**VIDS**）的网卡能够“插”在这个虚拟交换机上，从而拥有和物理机同一网段的 **IP** 地址，方便网络管理员通过管理网段远程登录这台虚拟机
给入侵检测系统（**IDS**）分配了 **500G** 的虚拟磁盘，用来存储抓包和安全日志
导入了厂商提供的 **VIDS** 系统镜像文件
IDS 的核心功能是“旁路监听”网络流量（目前可能更针对东西流量）

